codegraph_guard.sh — Companion Guide

Revision: 1 **Last modified:** 2026-09-24T12:00:53Z

Source: `constitution/scripts/codegraph/codegraph_guard.sh` (159 lines). Every statement below cites `codegraph_guard.sh:<line>`.

Overview

Mechanical gate `CM-CODEGRAPH-SAFE-INDEX-PATH` (line 3). It scans tracked scripts, hooks and documentation commands for two patterns that bypass the single safe writer entry `codegraph_safe.sh` (lines 5-8):

1. **`direct-stock-init-index`** — a direct stock `codegraph init` or `codegraph index`, including the `npx [-y] @colbymchenry/codegraph index` form and a quoted python argv-list form (`"codegraph", "index"`) (regex `CMD`, line 50).
2. **`masked-codegraph-failure`** — a `codegraph <subcommand>` line that also contains `|| true` or `|| :` (regexes `MASK_CMD` + `MASK`, lines 51-52).

Both patterns were at the root of the 2026-09-23 bulk-window incident: a legacy one-liner reached the bulk-window FK-cascade hazard and masked every failure (lines 9-10).

The scanner is read-only; it never runs `codegraph` itself.

Prerequisites

- `bash`, `python3` (stdlib only) (lines 25, 47).
- For `--self-test`: a writable `$TMPDIR` (default `/tmp`) and `codegraph_safe.sh` in the same directory (lines 119, 124).

Usage examples

```
# Default roots: this constitution's scripts/ dir, plus the consuming
# project's scripts/ dir when the constitution is its submodule
codegraph_guard.sh

# Explicit roots (repeatable)
codegraph_guard.sh --root <project_root>/scripts --root <project_root>/hooks

# Built-in golden-true / golden-false fixture check
codegraph_guard.sh --self-test

# Help (prints the header, lines 2-32)
codegraph_guard.sh --help
```

Flags (lines –)

Flag	Meaning
`--root DIR`	Add a scan root; repeatable. Missing value → exit 2 (line 38). A root that is not a directory → exit 2 (line 158).
`--self-test`	Run the fixture self-test instead of a scan (lines 117-150).
`-h`, `--help`	Print header lines 2-32 and exit 0 (line 42).

Any other argument → `unknown argument`, exit 2 (line 43).

Default roots (lines –)

When no `--root` is given: `$HERE/..` (the constitution `scripts/` dir). If the directory three levels up contains both `scripts/` and `constitution/`, its `scripts/` is added as a second root.

Output

One line per offender: `<relpath>:<line>: <rule>: <text>` (text truncated to 160 characters, lines 101-104), then a summary:

- `GUARD PASS files=<n> offenders=0` or `GUARD FAIL files=<n> offenders=<n>` (line 113);
- or `GUARD BLIND no candidate files under: <roots>` (line 110).

Exit codes

Code	Meaning	Source
0	Clean scan; `--help` ; `--self-test` all PASS	lines 42, 114, 148
1	Offenders found; `--self-test` had a failure	lines 114, 149
2	Usage error, root not a directory, or BLIND (0 candidate files under a root)	lines 38, 43, 111, 158

BLIND is deliberately exit 2: an empty scan proves nothing (header line 24).

What is scanned (lines – , –)

- Extensions: `.sh .bash .py .js .mjs .ts .yml .yaml .md` (line 53), plus extension-less files whose first two bytes are `#!` (lines 67-73).
- Skipped directories: `.git node_modules .codegraph __pycache__ .compressed out` (line 54). Symlinks and non-regular files are skipped (lines 62-63).
- **Markdown:** only lines inside ``` fences are scanned; prose is not a command (lines 91-96).
- **Other files:** full-line comments starting with `#` or `//` are skipped as carriers (lines 97-98).

- **Exemption:** a line containing `codegraph-guard: allow <reason>` is skipped; the reason must be non-empty (regex `ALLOW`, line 52; applied at line 99). A bare marker is not an exemption.
- Unreadable files are skipped without counting as an offender (lines 86-87) but still count toward the file total (line 81).

Edge cases

- A trailing comment such as `cmd # codegraph index …` on a code line is NOT skipped (only full-line comments are, line 97) — add an `allow` marker with a reason if the line is legitimate.
- `CMD` takes precedence: a line matching both rules reports only `direct-stock-init-index` (lines 101-104).
- The lookbehind `(?<![A-Za-z0-9_.-])` in `CMD` / `MASK_CMD` (lines 50-51) means names such as `codegraph_safe.sh index` or `my-codegraph index` do not match; the self-test asserts `codegraph_safe.sh` itself is never flagged (lines 142-145).
- `MASK` only matches `|| true` / `|| :` followed by a non-identifier character (line 52); other masking forms (`; true`, `set +e`) are not detected. UNCONFIRMED: whether any other masking idiom is used in the scanned trees.

Self-test (lines –)

Creates `$TMPDIR/cg_guard_selftest/run.XXXXXX` with `scripts/` + `docs/`, copies `codegraph_safe.sh` in, then checks four cases:

1. golden-false-with-carrier: a comment and a prose mention → exit 0;
2. golden-true: planted `codegraph index` → exit 1 and reported at `scripts/planted.sh:3`;
3. golden-true: planted `(codegraph sync . || true)` → reported at `scripts/mask.sh:2`;
4. the copied launcher is not flagged.

The run directory is removed afterwards (line 147); the parent `cg_guard_selftest` directory is left in place.

Related scripts

Guide	Relation
codegraph_safe	The only permitted writer entry this gate protects
codegraph_safe_helper	Helper used by the writer entry
disk_tripwire	Free-space watcher started by the writer entry
index_watch	Progress-proven watchdog
fk_index_patch	Runner patcher
fk_cascade_probe	Bulk-window hazard probe
codegraph_runner_patches	Version-keyed runner patches
codegraph_scope_guard	Scope proof
scope_render	Scope generator
codegraph_mcp_probe	MCP reachability probe
lumen_verify	Semantic-index health check
gen_lumenignore	Semantic-index scope generator
codegraph_lumen_overview	Overview

Tests: `scripts/codegraph/tests/test_guard.sh` exists (header line 26). UNCONFIRMED: its content was not read for this guide.

Last verified

2026-09-24 — read `codegraph_guard.sh` lines 1-159 in full; `bash -n codegraph_guard.sh` passed. The script (including `--self-test`) was NOT executed for this guide.